

Qwen3 4B Instruct - MAI Suitability Report

Qwen3 4B Instruct — Local Model Suitability Report

Benchmark Evaluation for MAI’s Local LLM Role

Overall Verdict: PASS Strong candidate for MAI’s local LLM

This evaluation is based on results from an i5-1235U + 16 GB RAM, CPU-only setup, and is judged strictly against MAI’s requirements rather than general model quality.

1. Comparison vs. Qwen3 8B

Metric	Qwen3 8B	Qwen3 4B Instruct
First response	~25 sec	2-4 sec
Typical completion	~1 min	4-8 sec
Thinking overhead	High	Avoided
Structured output	Not fully tested	Very good
MAI suitability	CAUTION	PASS Strong candidate

For the target hardware, Qwen3 4B Instruct is much closer to what’s needed.

2. Detailed Test Results

Test 1 — Friendly Conversation

Result: PASS Pass | **Speed:** PASS Excellent — 2s start, 7s completion | **Score:** 9/10

“I get it — feeling overwhelmed is totally normal...”

Correctly understood emotional state, the user’s problem, the need for prioritization, and the requested tone (friendly, practical, concise).

Minor issue: the model said “Hey there, sir!” — MAI should not let the model randomly address the user as “sir”; this isn’t a model failure and will be controlled via the MAI system prompt/personality.

Test 2 — Intent Classification

Result: PASS Excellent | **Speed:** 2s / 4s | **Score:** 10/10

```
{
  "intent": "task_retrieval",
  "needs_tool": true,
  "tool": "calendar_retrieval",
  "complexity": "simple"
}
```

Correctly identified: user intent (retrieve tasks), that a tool was needed, the appropriate tool category (calendar/tasks), and complexity (simple). It also obeyed the “Return ONLY valid JSON” instruction.

Test 3 — Tool Routing

Result: PASS Excellent | **Speed:** 3s / 7s | **Score:** 10/10

Correctly chose "action": "calculator", demonstrating the model can act as a reliable tool router for mathematical requests.

Test 4 — Current Information Detection

Result: PASS Critical pass | **Speed:** 3s / 5s | **Score:** 10/10

Chose "action": "web_search" instead of hallucinating an answer to a “what happened this week” style query. It correctly recognized that the word “latest” signals a need for current information requiring web search — exactly the behavior needed for an agentic assistant.

Test 5 — Long-Term Memory Extraction

Result: PASS Excellent | **Speed:** 3s / 7s | **Score:** 10/10

Correctly extracted: - **Goal** → Preparing for software placements - **Preference** → Prefers C++ for DSA - **Project** → Building MAI

It correctly ignored transient information (“Today I studied for three hours”), proving it can distinguish potentially persistent information from temporary conversation details.

Future improvement: expand memory types to include preference, goal, project, fact, relationship, skill, and constraint categories.

Test 6 — Entity and Relationship Extraction

Result: PASS Very good | **Speed:** 3s / 7s | **Score:** 9/10

Correctly identified entities:

Entity	Type
Sri	PERSON
MAI	PROJECT
Python	TECHNOLOGY
FastAPI	TECHNOLOGY
PostgreSQL	DATABASE

And relationships: Sri → building → MAI; MAI → uses → PostgreSQL; MAI → built with → Python; MAI → built with → FastAPI.

Improvement needed: normalize relationship labels for Neo4j — use BUILDING, USES, USES_TECHNOLOGY instead of free-text phrases like “is building” / “uses” / “is built with”, e.g.:

```
(:Person)-[:BUILDING]->(:Project)
(:Project)-[:USES_TECHNOLOGY]->(:Technology)
(:Project)-[:USES_DATABASE]->(:Database)
```

This is a schema issue, not a model problem.

Test 7 — Ambiguity and Hallucination Prevention

Result: PASS Excellent | **Speed:** 4s / 6s | **Score:** 10/10

The model correctly said it didn’t know what “that thing” referred to and asked for clarification instead of inventing an answer.

```
Missing information → MAI recognizes uncertainty → Ask user for clarification
```

rather than guessing and taking the wrong action. This is extremely valuable for MAI.

Test 8 — Instruction Following

Result: PASS Pass | **Speed:** 2s / 4s | **Score:** 9/10

Prompt required exactly one sentence, maximum 15 words. Response: “An API is a set of rules for building and communicating between software.” — constraints were followed. Points deducted only for phrasing quality, not instruction-following failure.

Test 9 — Structured Output and Time Extraction

Result: PASS Excellent | **Speed:** 3s / 7s | **Score:** 10/10

Correctly extracted intent (`remind_me`), that a tool was needed, the tool (`reminder`), and the time expression (“tomorrow at 6 PM”) — well suited to MAI’s first-stage intent pipeline.

Architecture note: the LLM should extract the raw phrase (“tomorrow at 6 PM”), while application code — not the LLM — should convert it into an exact timestamp based on timezone:

```
LLM → "tomorrow at 6 PM" → Date/time service → 2026-09-01T18:00:00+05:30
```

Never trust an LLM to perform critical date calculations itself.

Test 10 — Multi-Intent Request

Result: PASS Good, with one small disagreement | **Speed:** 2s / 7s | **Score:** 9/10

Correctly identified the sub-intents `check_calendar`, `identify_most_important_meeting`, and `create_reminder`, the tools `calendar` and `reminder`, and set `requires_multiple_steps: true`.

Disagreement: the model classified this as `complex`; a `medium` classification would be more accurate, since the task is multi-step but not deeply complex reasoning. Not a serious failure — complexity categories are subjective without a precise definition. Proposed definitions for later:

Level	Definition
Simple	One direct action
Medium	Multiple actions or moderate reasoning
Complex	Advanced reasoning, long planning, uncertainty, many dependencies

Test 11 — Local vs. Cloud Routing

Result: PASS Excellent | **Speed:** 3s / 8s | **Score:** 10/10

Correctly routed a complex architecture/scalability question (multi-agent systems, failure recovery, long-term memory, millions of users, comparison of three approaches) to `route: cloud` — recognizing it should not attempt to handle everything locally.

3. Final Score Summary

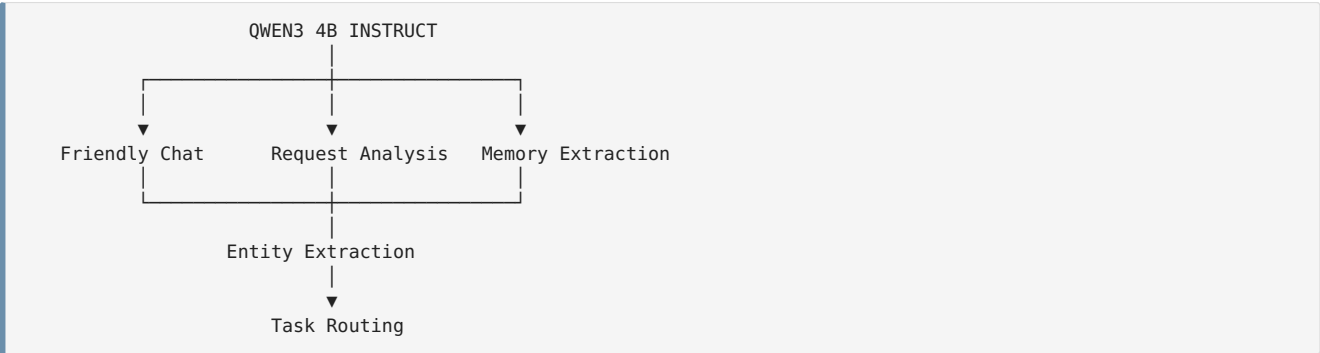
Capability	Score	Verdict
Friendly conversation	9/10	PASS
Intent classification	10/10	PASS
Tool routing	10/10	PASS
Current info detection	10/10	PASS
Memory extraction	10/10	PASS
Entity extraction	9/10	PASS
Ambiguity handling	10/10	PASS
Instruction following	9/10	PASS
Structured JSON	10/10	PASS
Multi-step analysis	9/10	PASS
Local/cloud routing	10/10	PASS

Overall: 96/110 ≈ 8.7/10
MAI suitability rating (for this hardware): 9/10

4. Conclusion

Qwen3 4B Instruct has passed the initial MAI evaluation and is selected as the current local model candidate for MAI v1.

Its proposed role in the architecture:



Important caveat: it has passed the initial benchmark, but has not yet proven to be the final “perfect” model. Real robustness tests are still required before locking in the decision.

5. Tests Still Missing

1. Repeated JSON Reliability

Run the same structured task multiple times (10 runs) and confirm valid JSON every time — a single successful response doesn’t guarantee production reliability.

2. Difficult / Ambiguous Routing

Example: “Find information about the best laptops and help me choose one based on my needs.” Should MAI use web search, cloud reasoning, or both? Test whether it makes sensible decisions when the boundary is unclear.

3. False Tool Calls

Example: “Explain what a REST API is.” It should choose `respond_directly` rather than unnecessarily calling web search —

avoiding a pattern where every request triggers a slow, expensive tool call.

4. Multi-Turn Context

Example conversation:

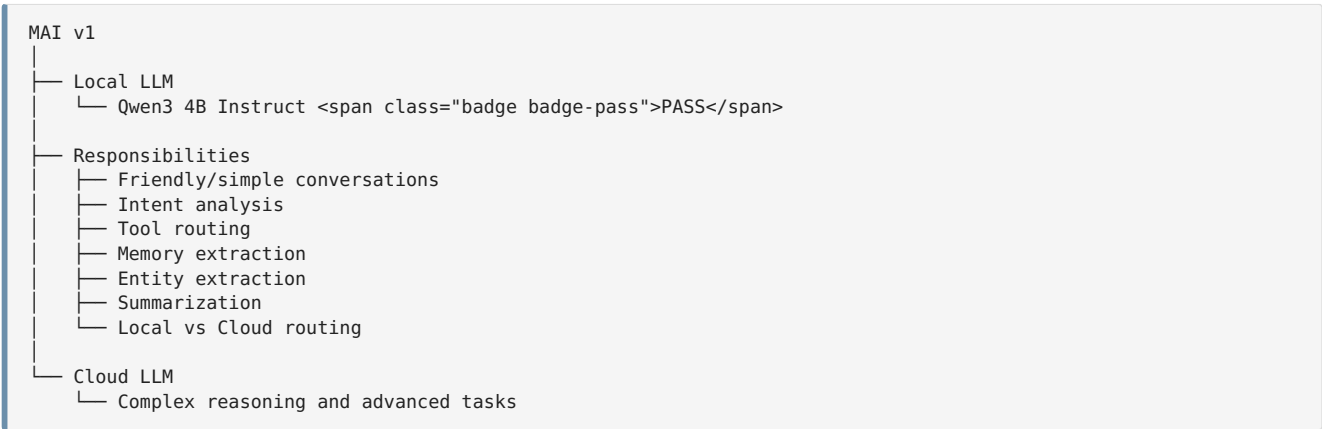
```
User: I'm building MAI.  
MAI: Great!  
User: I want to use it locally.  
MAI: Okay.  
User: Which model should I use for it?
```

Does the model correctly resolve what “it” refers to across turns?

5. Stress Test

Provide a longer prompt containing multiple instructions, irrelevant information, and conflicting information, and check whether the model can still extract the correct structured result.

6. Recommended Architecture (Provisional)



This is a architecture decision grounded in actual on-device testing. The recommended next step is to complete the robustness tests above before moving on to further model comparisons or backend development — to confirm not just that Qwen3 4B can perform well once, but that it is reliable enough to serve as a production component in a real agent system.